

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME:
ARTIFICIAL
INTELLIGENCE AND
EXPERT SYSTEMS
OUTCOME COVERED
COURSE CODE: MLA01

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 90 Mins No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

*I **Nikhil Kumar Reddy Kadiveti (192425031)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given RomaniaMap Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning
5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

1) Greedy Best-First Search

1. Problem Statement

Find a path from a source city to a goal city using Greedy Best-First Search.

2. Problem Analysis

Initial state: **Arad**; Goal state: **Bucharest**.

3. AI Technique Selection

Greedy Best-First Search is selected as an informed search technique.

4. Justification

It selects the node that appears closest to the goal using a heuristic.

5. State Space Representation

Cities are nodes and roads are edges in the Romania road graph.

6. Variables, Domains, and Constraints

Not Applicable, as this is an informed search problem.

7. Heuristic Function

$h(n)$ is the straight-line distance from node n to Bucharest.

8. Algorithm Design / Pseudocode

1. Insert the source node into OPEN.
2. Select the node with the lowest $h(n)$.
3. If it is the goal, return the path.
4. Otherwise, expand it and add successors.
5. Repeat until the goal is reached or OPEN is empty.

9. Flowchart / Algorithm Diagram

Start → Insert Source → Lowest $h(n)$ → Expand → Goal → Path

10. Working / Execution Trace

Arad → Sibiu → Fagaras → Bucharest

11. Complexity Analysis

Time: $O(b^m)$ | Space: $O(b^m)$

12. Advantages and Limitations

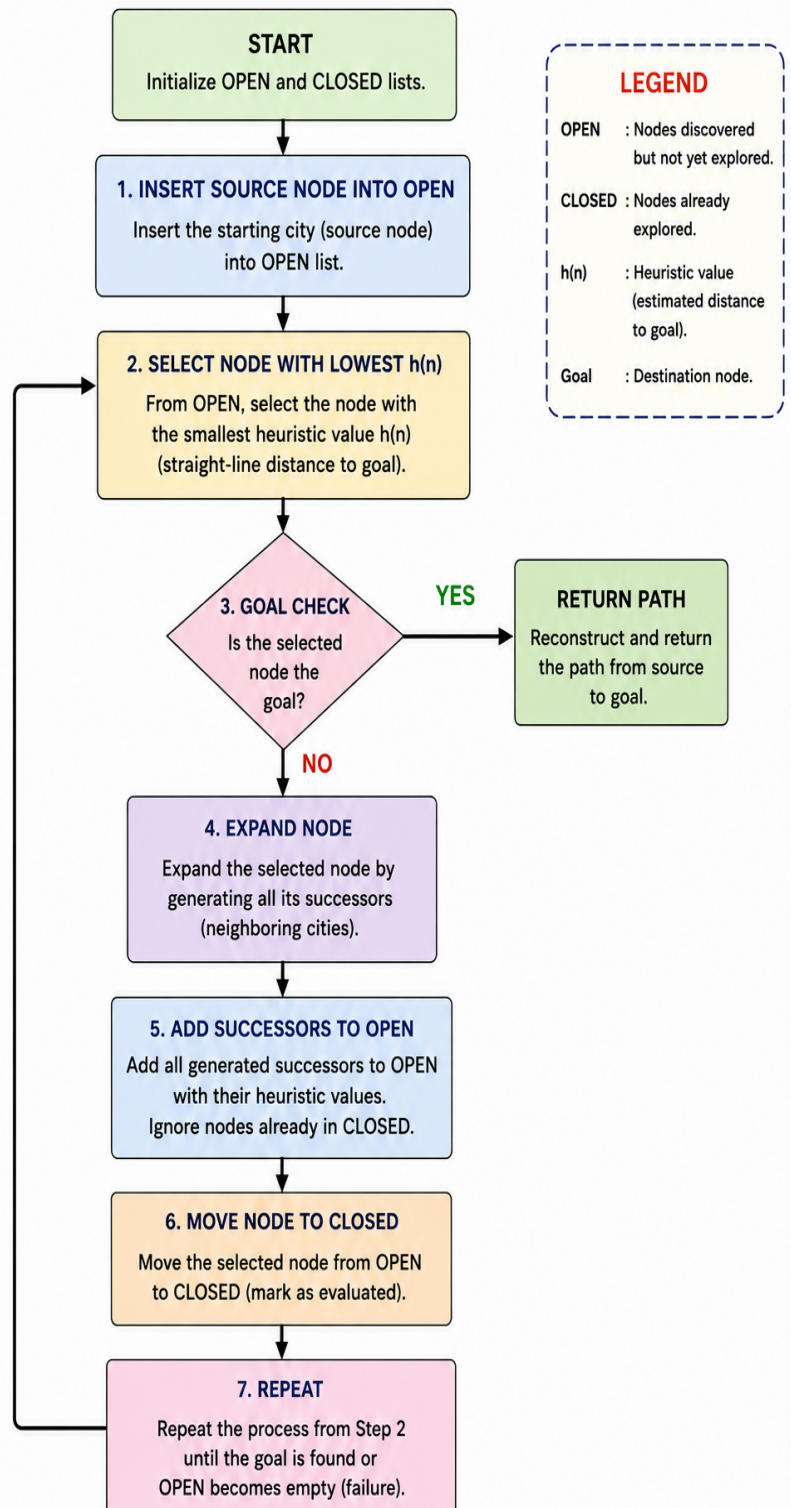
Advantage: Fast goal-directed search.

Limitation: Not always optimal.

13. Conclusion

Greedy Best-First Search quickly finds a goal-directed path using heuristic information.

GREEDY BEST-FIRST SEARCH ALGORITHM



2) A* Search

1. Problem Statement

Find the optimal path for a delivery robot through a weighted road network.

2. Problem Analysis

Initial state: Warehouse; Goal state: Customer.

3. AI Technique Selection

A* is selected for optimal path finding.

4. Justification

It considers both the travelled cost and estimated remaining cost.

5. State Space Representation

Locations are nodes and roads are weighted edges.

6. Variables, Domains, and Constraints

Not Applicable, as this is a path-search problem.

7. Heuristic Function

$f(n) = g(n) + h(n)$, where $g(n)$ is actual cost and $h(n)$ is estimated cost.

8. Algorithm Design / Pseudocode

1. Insert the start node into OPEN with $g(\text{start})=0$.
2. Select the node with minimum $f(n)=g(n)+h(n)$.
3. If it is the goal, return the path.
4. Expand it and calculate $g(n)$, $h(n)$ and $f(n)$.
5. Update OPEN and repeat until the goal is reached.

9. Flowchart / Algorithm Diagram

Start → Calculate $f(n)$ → Minimum $f(n)$ → Expand → Goal → Optimal Path

10. Working / Execution Trace

Warehouse → A → B → Customer, selecting the node with minimum $f(n)$.

11. Complexity Analysis

Time: $O(b^d)$ | Space: $O(b^d)$

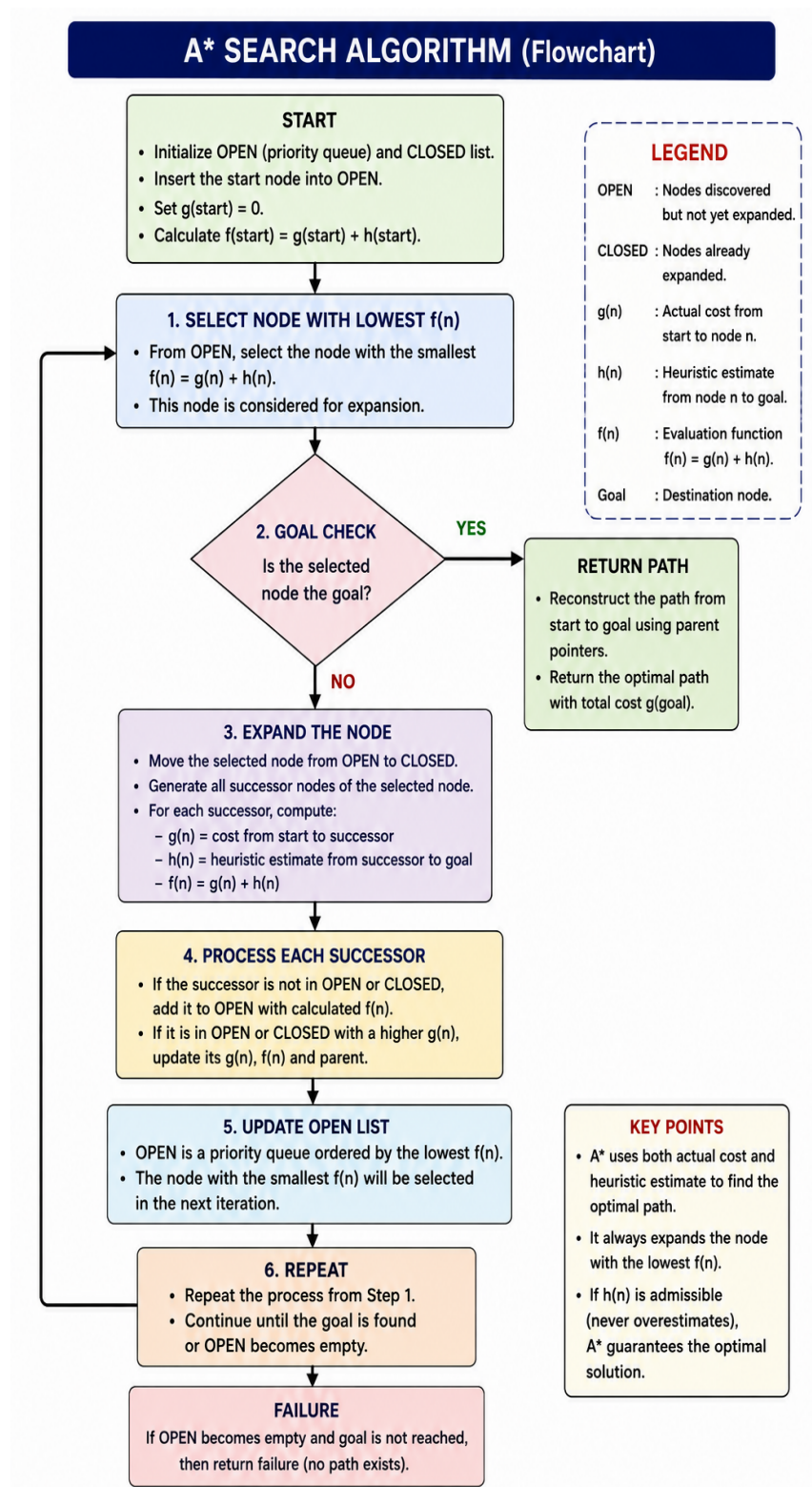
12. Advantages and Limitations

Advantage: Can find an optimal path.

Limitation: High memory usage.

13. Conclusion

A* efficiently finds an optimal path by combining actual and estimated costs.



3) CSP, MRV and Forward Checking

1. Problem Statement

Create an examination timetable without scheduling conflicting subjects together.

2. Problem Analysis

Initial state: **Unassigned subjects**; Goal state: **Conflict-free timetable**.

3. AI Technique Selection

CSP with MRV and Forward Checking is selected.

4. Justification

CSP directly represents subjects, time slots and scheduling constraints.

5. State Space Representation

Subjects are variables and available examination slots are their domains.

6. Variables, Domains, and Constraints

Variables: Subjects; **Domains:** Time slots;
Constraint: Conflicting subjects cannot share a slot.

7. Heuristic Function

Not Applicable, as CSP uses MRV instead of an informed-search heuristic.

8. Algorithm Design / Pseudocode

1. Select an unassigned subject using MRV.
2. Assign a valid time slot.
3. Apply Forward Checking to conflicting subjects.
4. If a domain becomes empty, backtrack.
5. Repeat until all subjects are assigned.

9. Flowchart / Algorithm Diagram

Start → **MRV** → **Assign Slot** → **Forward Check** → **Conflict?** → **Backtrack/Continue**

10. Working / Execution Trace

MRV selects the subject with the fewest slots, then Forward Checking removes conflicting choices.

11. Complexity Analysis

Time: $O(d^n)$ | **Space:** $O(n)$

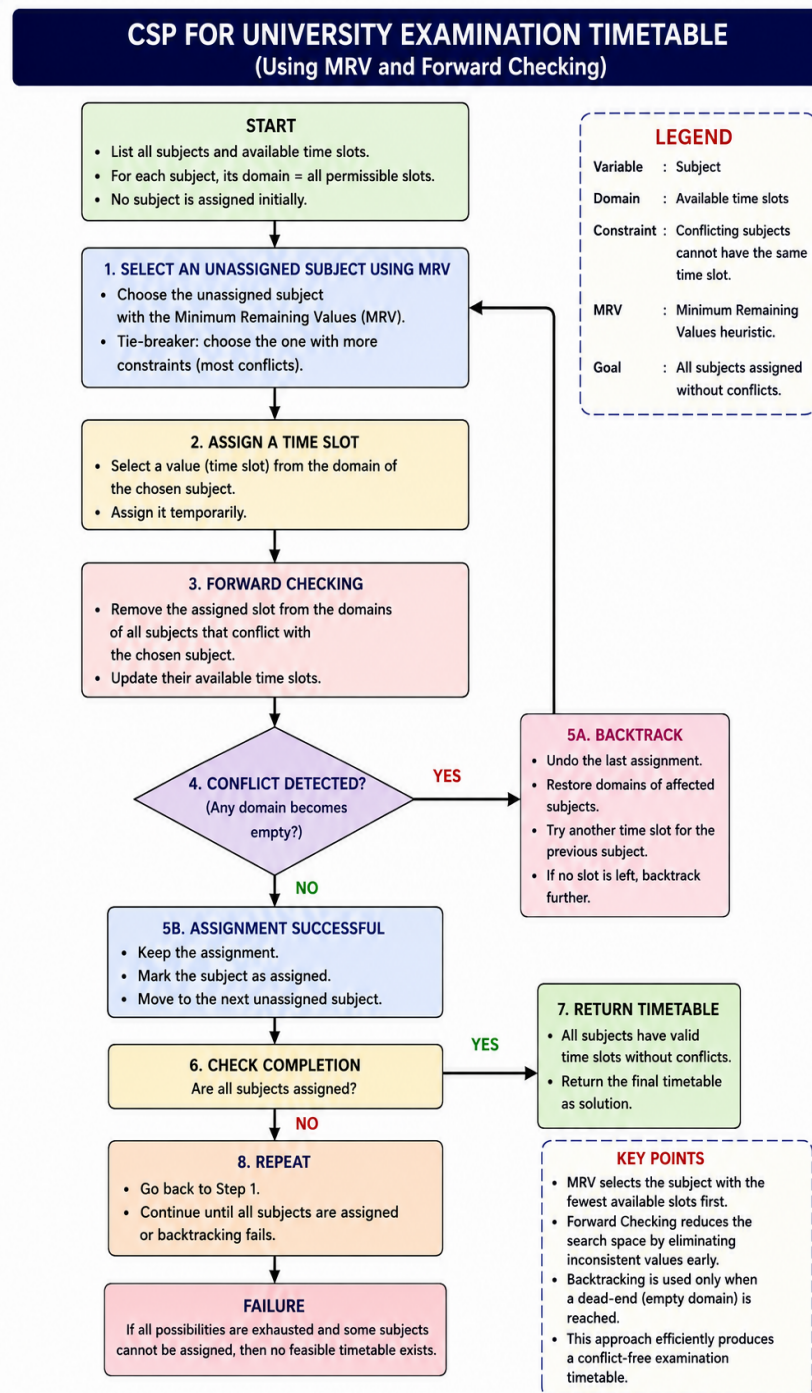
12. Advantages and Limitations

Advantage: Reduces unnecessary assignments.

Limitation: May require backtracking.

13. Conclusion

CSP with MRV and Forward Checking produces a valid conflict-free timetable efficiently.



4) Minimax and Alpha-Beta Pruning

1. Problem Statement

Find the best move for the MAX player using Minimax and Alpha-Beta pruning.

2. Problem Analysis

Initial state: **Root node**; Goal state: **Best move for MAX**.

3. AI Technique Selection

Minimax with Alpha-Beta Pruning is selected.

4. Justification

Minimax evaluates competing moves while Alpha-Beta removes unnecessary branches.

5. State Space Representation

The game is represented as a tree containing MAX, MIN and terminal nodes.

6. Variables, Domains, and Constraints

Not Applicable, as this is a game-tree search problem.

7. Heuristic Function

Not Applicable because terminal utility values are directly evaluated.

8. Algorithm Design / Pseudocode

1. Start from the root of the game tree.
2. MAX selects the maximum and MIN selects the minimum value.
3. Evaluate terminal nodes and propagate values upward.
4. Update Alpha and Beta during evaluation.
5. Prune when $\alpha \geq \beta$ and select MAX's best move.

9. Flowchart / Algorithm Diagram

Start → **Evaluate Tree** → **MAX/MIN** → **Alpha-Beta Check** → **Prune** → **Best Move**

10. Working / Execution Trace

For values (3,5) and (2,9): MIN values are **3** and **2**, so MAX selects **3**.

11. Complexity Analysis

Minimax: $O(b^d)$ | **Alpha-Beta best case:** $O(b^{(d/2)})$

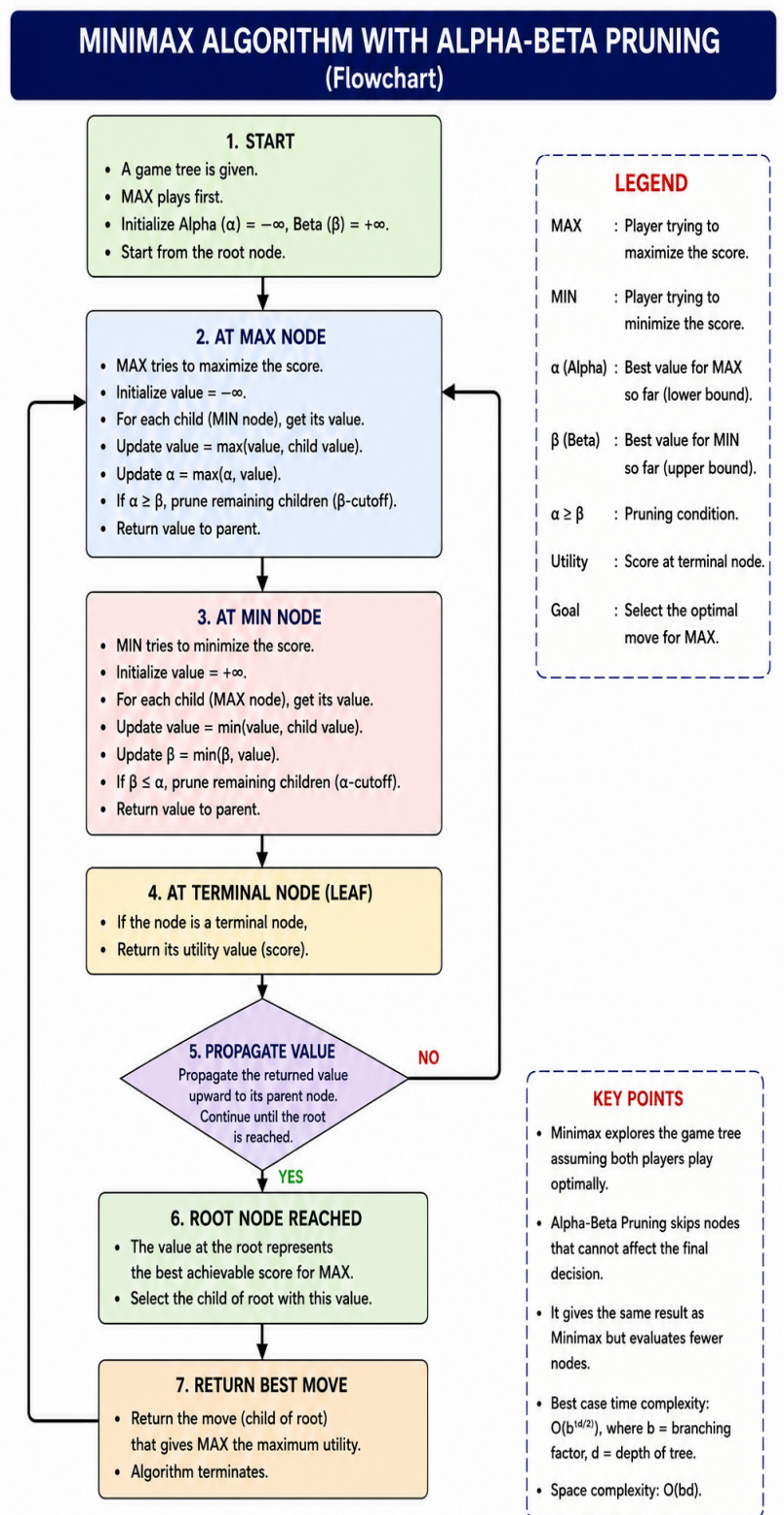
12. Advantages and Limitations

Advantage: Reduces node evaluation.

Limitation: Depends on node ordering.

13. Conclusion

Alpha-Beta pruning improves Minimax efficiency while preserving the optimal decision.



5) Online Search Agent

1. Problem Statement

Design a search strategy for a warehouse robot facing unexpected obstacles.

2. Problem Analysis

Initial state: **Robot position**; Goal state: **Destination**.

3. AI Technique Selection

Online Search Agent is selected for dynamic navigation.

4. Justification

It observes environmental changes and adapts the route during execution.

5. State Space Representation

Warehouse positions form states and available movements form actions.

6. Variables, Domains, and Constraints

Variables: Robot positions; **Domains:** Possible moves; **Constraint:** Avoid obstacles.

7. Heuristic Function

Manhattan distance can estimate the remaining distance to the goal.

8. Algorithm Design / Pseudocode

1. Start the robot at the initial position.
2. Observe the environment and possible moves.
3. Select the best safe move toward the goal.
4. If an obstacle appears, recalculate the route.
5. Repeat until the robot reaches the goal.

9. Flowchart / Algorithm Diagram

Start → **Observe** → **Select Safe Move** → **Move** → **Obstacle?** → **Recalculate** → **Goal**

10. Working / Execution Trace

The robot follows the selected route and changes direction whenever a new obstacle appears.

11. Complexity Analysis

Time: $O(N)$ | **Space:** $O(N)$ for N reachable states.

12. Advantages and Limitations

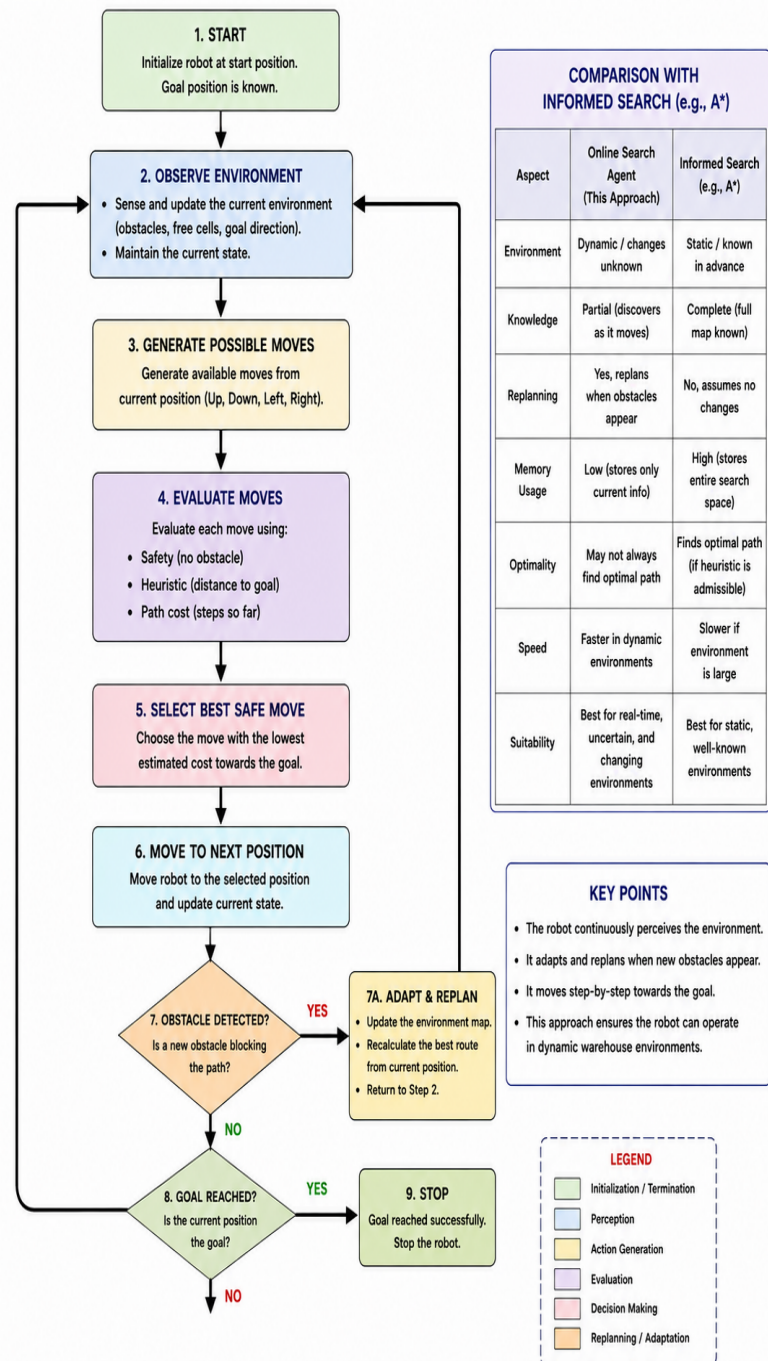
Advantage: Adapts to dynamic obstacles.

Limitation: Replanning increases execution time.

13. Conclusion

Online search enables the robot to dynamically adapt its path to environmental changes.

ONLINE SEARCH AGENT FOR WAREHOUSE ROBOT (Flowchart)



EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well- labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.